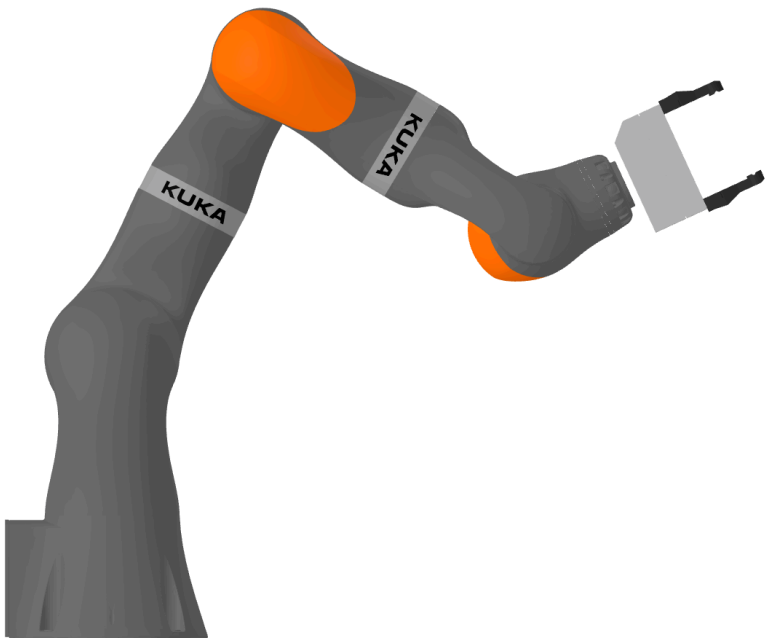
Basic Kinematics



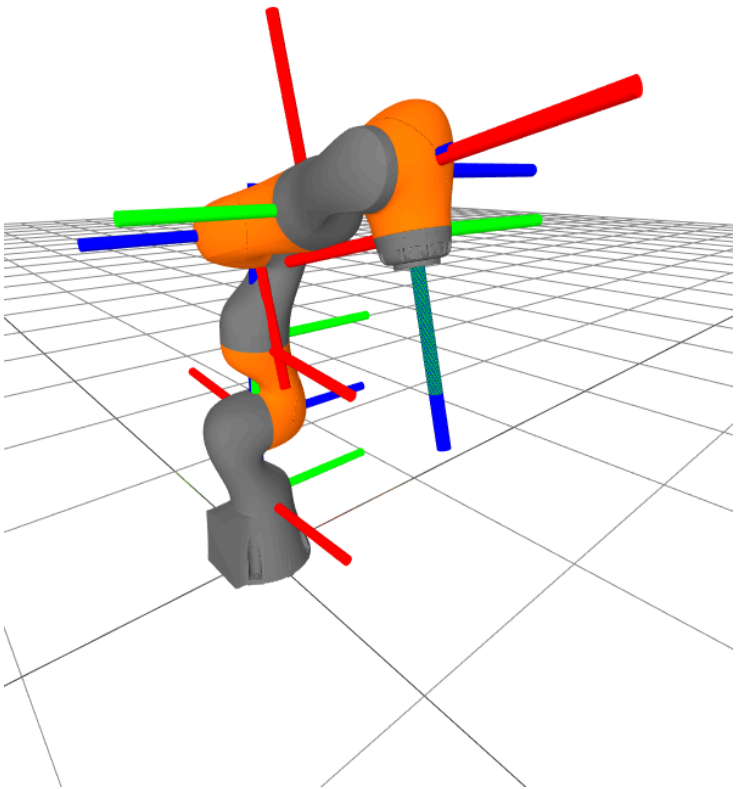
MIT 6.4210/2: Robotic Manipulation

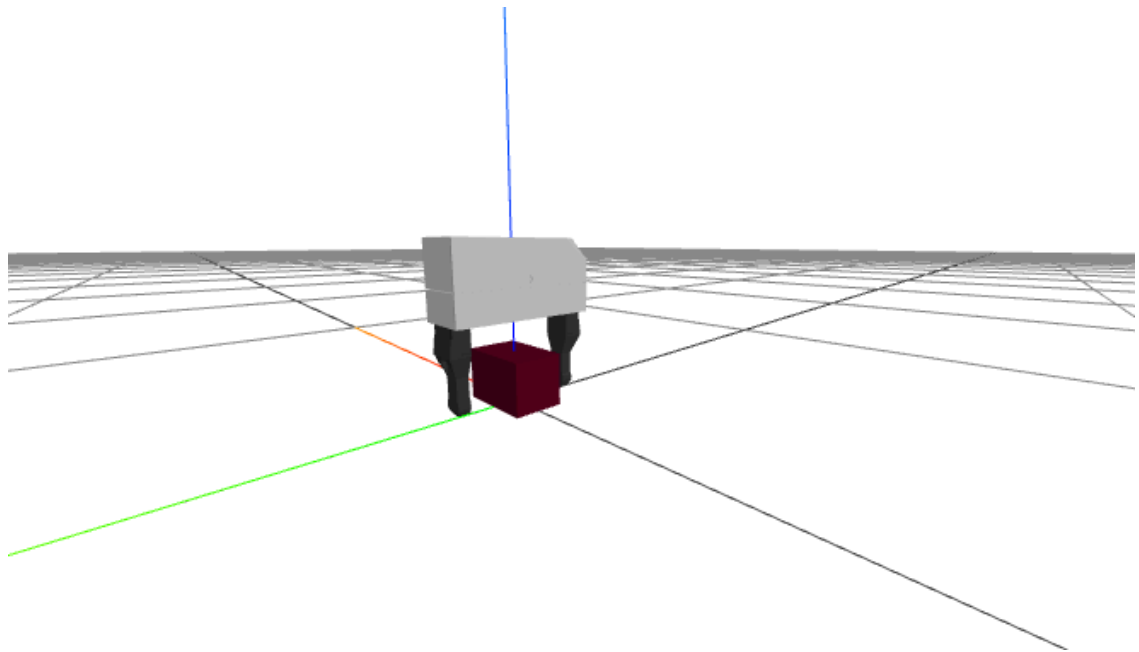
Fall 2026, Lecture 4

Massachusetts Institute of Technology

Basic Pick and Place

Step 1: Kinematic Frames / Spatial Algebra



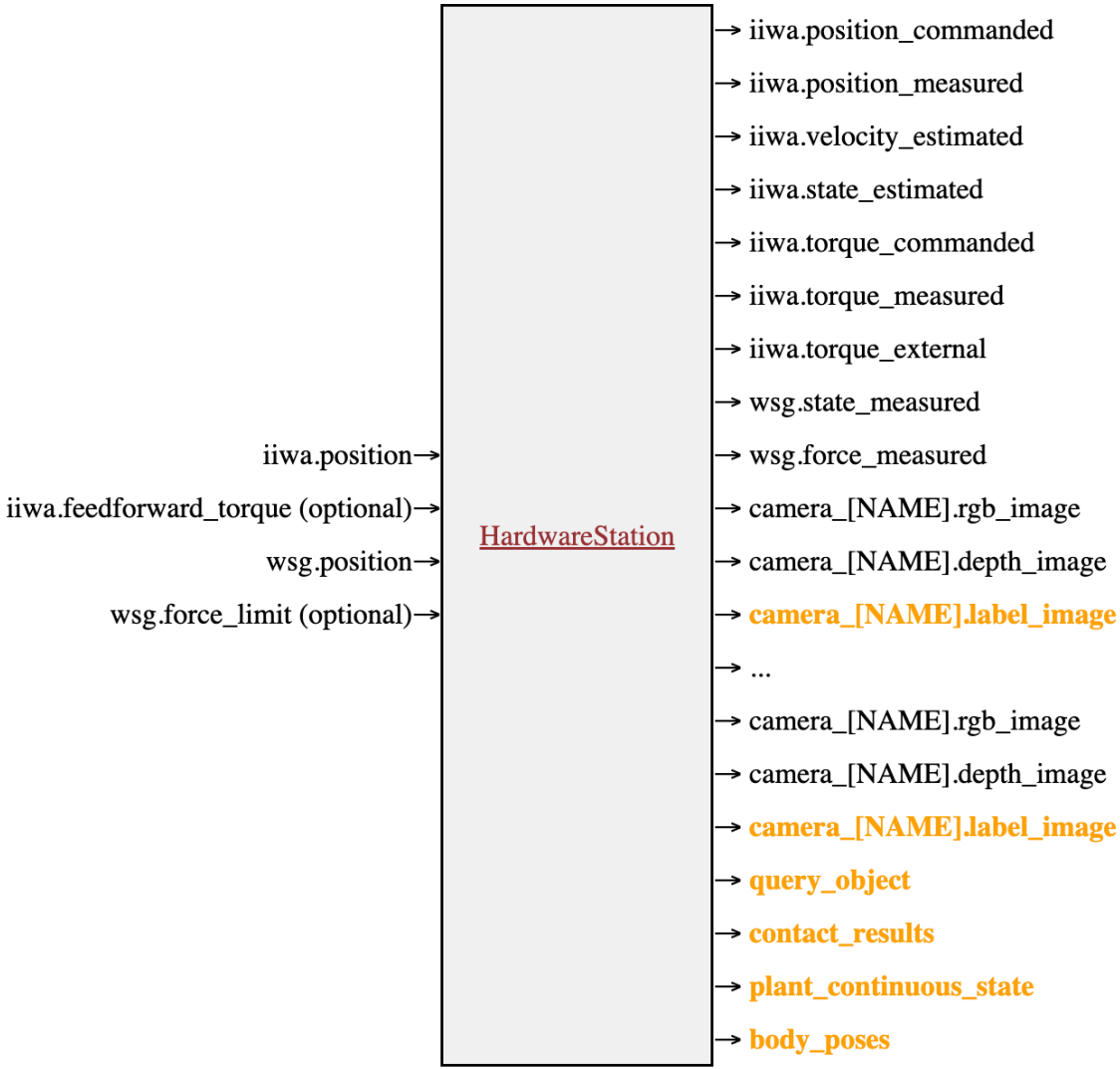


Step 2: Gripper Frame Plan "Sketch"

https://manipulation.mit.edu/data/pick_sketch.html

Gripper trajectory

- Given a sequence of transforms (gripper poses)
- Generate continuous trajectory (function of time) via PiecewisePose
  - piecewise quaternion SLERP (spherical linear interpolation) for orientation
  - piecewise polynomial for position


iiwa.position
iiwa.feedforward_torque (optional)
wsg.position
wsg.force_limit (optional)
HardwareStation
iiwa.position_commanded
iiwa.position_measured
iiwa.velocity_estimated
iiwa.state_estimated
iiwa.torque_commanded
iiwa.torque_measured
iiwa.torque_external
wsg.state_measured
wsg.force_measured
camera_[NAME].rgb_image
camera_[NAME].depth_image
camera_[NAME].label_image
...
camera_[NAME].rgb_image
camera_[NAME].depth_image
camera_[NAME].label_image
query_object
contact_results
plant_continuous_state
body_poses

Kinematics - relevant spaces

- Configuration space / joint space
  - A vector of the joint "angles" (positions)
  - "minimal coordinates"
- Workspace / operational space / task space
  - Spatial representation of the poses of the links of the robot - typically the "end-effector" link
  - "maximal coordinates"

Kinematics

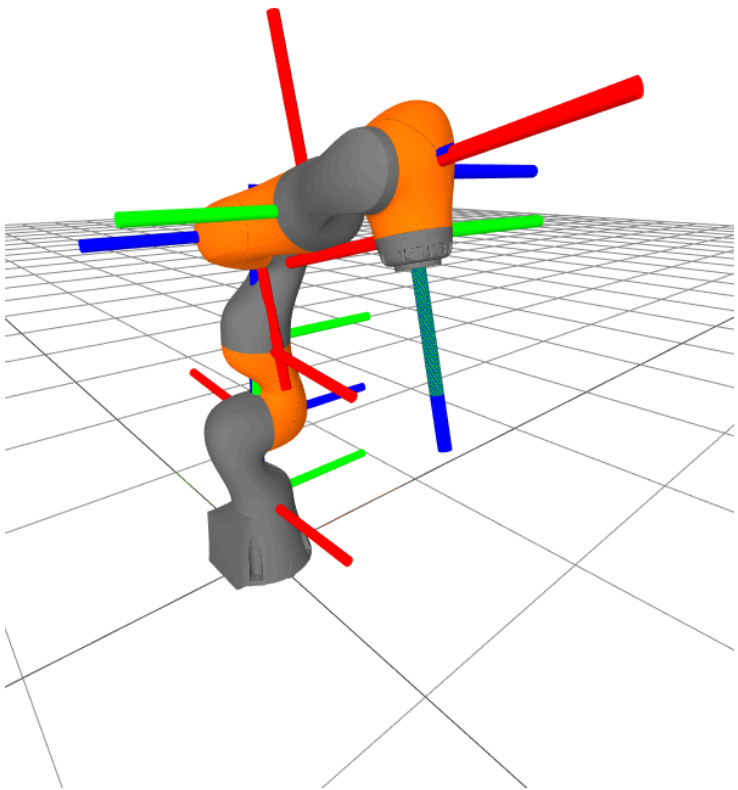
- *Forward kinematics*: joint positions ⇒ pose $q \Rightarrow X^B$
- ***Inverse** kinematics**: pose ⇒ joint positions $X^B \Rightarrow q$

- *Differential kinematics*:
  joint positions, velocities ⇒ spatial velocity $q, v \Rightarrow V^B$

- *Differential **inverse** kinematics*:
  spatial velocity, joint positions ⇒ joint velocities

$$V^B, q \Rightarrow v$$

Forward Kinematics



$$X^B = f_{kin}^B(q)$$

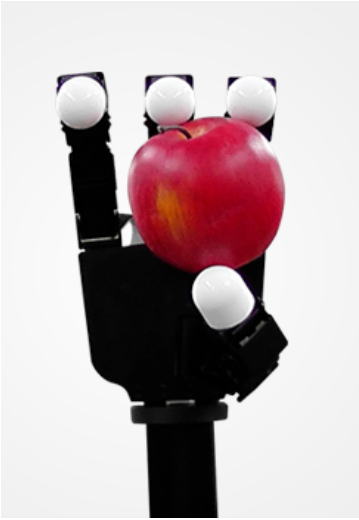
Forward Kinematics

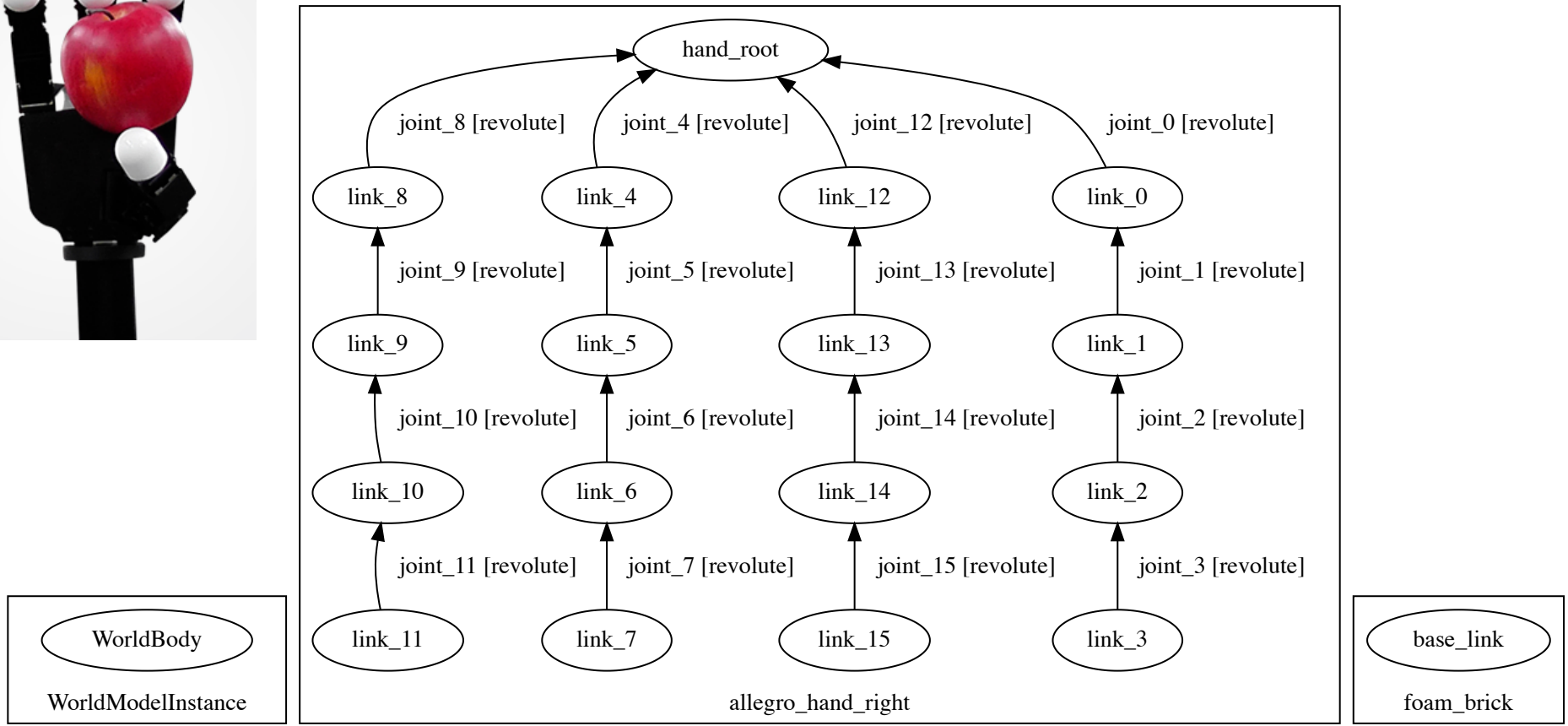
$$ {}^{P}\boldsymbol{X}^{C}(q) = {}^{P}\boldsymbol{X}^{J_P} {}^{J_P}\boldsymbol{X}^{J_C}(q) {}^{J_C}\boldsymbol{X}^{C}. $$

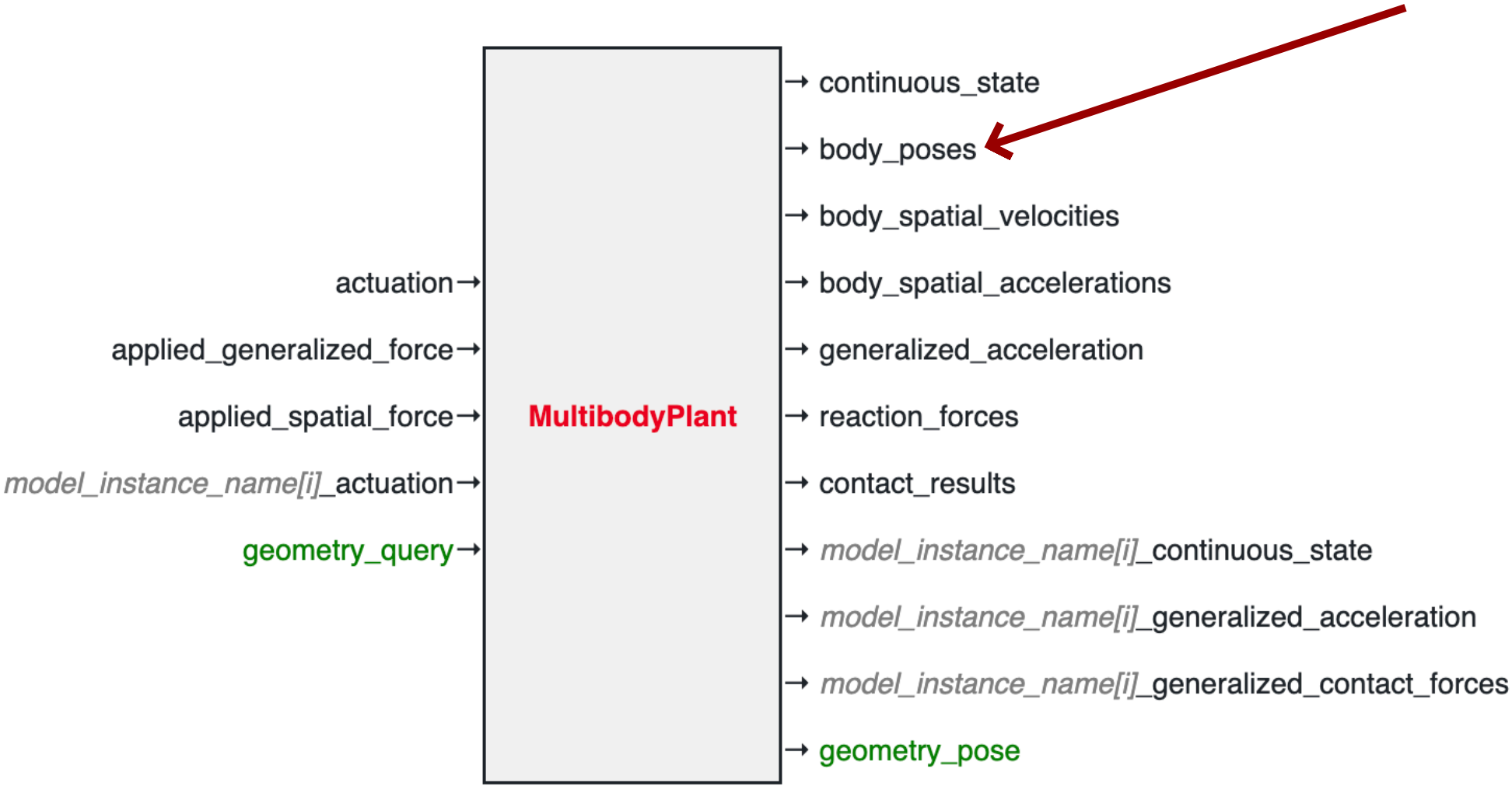
Simplify by choosing link frame at the joint

Robot Description Files

```
<?xml version="1.0"?>
<sdf version="1.7">
  <model name="iiwa14">
    <link name="iiwa_link_0">
      <inertial>
        <pose>-0.1 0 0.07 0 0 0</pose>
        <mass>5</mass>
        <inertia>
          <ixx>0.05</ixx>
          <ixy>0</ixy>
          <ixz>0</ixz>
          <iyy>0.06</iyy>
          <iyz>0</iyz>
          <izz>0.03</izz>
        </inertia>
      </inertial>
      <visual name="iiwa_link_0_fixed_joint_lump__iiwa_link_0_visual">
        <geometry>
          <mesh>
            <scale>1 1 1</scale>
            <uri>../meshes/visual/link_0.obj</uri>
```




hand_root
joint_8 [revolute]
joint_4 [revolute]
joint_12 [revolute]
joint_0 [revolute]
link_8
link_4
link_12
link_0
joint_9 [revolute]
joint_5 [revolute]
joint_13 [revolute]
joint_1 [revolute]
link_9
link_5
link_13
link_1
joint_10 [revolute]
joint_6 [revolute]
joint_14 [revolute]
joint_2 [revolute]
link_10
link_6
link_14
link_2
joint_11 [revolute]
joint_7 [revolute]
joint_15 [revolute]
joint_3 [revolute]
link_11
link_7
link_15
link_3
allegro_hand_right
WorldBody
WorldModelInstance
base_link
foam_brick


actuation
applied_generalized_force
applied_spatial_force
model_instance_name[i]_actuation
geometry_query
MultibodyPlant
continuous_state
body_poses
body_spatial_velocities
body_spatial_accelerations
generalized_acceleration
reaction_forces
contact_results
model_instance_name[i]_continuous_state
model_instance_name[i]_generalized_acceleration
model_instance_name[i]_generalized_contact_forces
geometry_pose

https://drake.mit.edu/doxygen_cxx/classdrake_1_1multibody_1_1_multibody_plant.html#aea0f8aca5f15e90f11e14602b3ba7068

Check yourself: A single "free" body

```
plant = MultibodyPlant(time_step = 0.0)
Parser(plant).AddModelFromFile(FindResourceOrThrow(
        "drake/examples/manipulation_station/models/061_foam_brick.sdf"))
plant.Finalize()
brick = plant.GetBodyByName("base_link")

context = plant.CreateDefaultContext()
```

What is q ? `q = plant.GetPositions(context)`

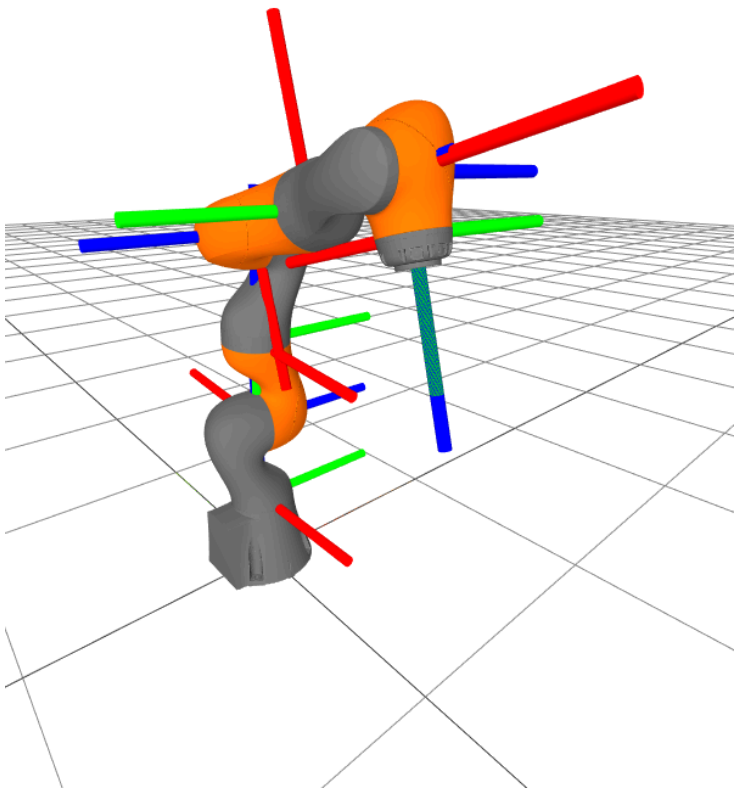
What is X^B ? `X_B = plant.EvalBodyPoseInWorld(brick, context)`

$$X^B = f^B_{kin}(q)$$

Inverse Kinematics

- Different versions (tasks), e.g. position vs pose of eef
- Difficult in general - solving trigonometric equations
  - There may be no solutions, multiple solutions or infinite solutions
- Analytic methods
  - For non-redundant robots - compute all solutions
- Numerical solution
  - Given initial guess at configuration, do e.g. numerical root finding
  - (More later)

Step 4: Differential Kinematics



$$X^B = f^B_{kin}(q)$$

$$dX^B = \frac{\partial f^B_{kin}(q)}{\partial q} dq$$

$$= \underbrace{J^B(q)}dq$$

Kinematic "**Jacobian**"

Spatial Velocity

$$\frac{d}{dt}\,{}^{A}\boldsymbol{X}_{C}^{B} \equiv {}^{A}\boldsymbol{V}_{C}^{B} = \begin{bmatrix} {}^{A}\boldsymbol{\omega}_{C}^{B} \\ {}^{A}\mathbf{v}_{C}^{B} \end{bmatrix}$$

angular velocity

translational velocity

note the typesetting

Q: How do we represent angular velocity, ω^B??

A: Unlike 3D rotations, here 3 numbers are sufficient and efficient; so we use the same representation everywhere.

$$(w_x, w_y, w_z)$$

Spatial Velocity Algebra

https://manipulation.csail.mit.edu/pick.html#jacobian

Computing Geometric Jacobian

Write $\hat{a}_i$ for the axis of joint i, a unit vector fixed in the joint frame J_i. A direction has no measured-from frame, only an expressed-in one, so we need just the one subscript: $\hat{a}_{i,W} = {}^{W}R^{J_i}(q)\,\hat{a}_i$ is that axis expressed in the world.

For a **revolute** joint the angular part is the axis itself, and the gripper origin sweeps a circle around it at $\mathrm{v} = \omega \times r$:

$$\left[{}^{W}J^{G}(q)\right]_{:,i} = \begin{bmatrix} \hat{a}_{i,W} \\ \hat{a}_{i,W} \times {}^{J_i}p_W^G \end{bmatrix}$$

The lever arm ${}^{J_i}p_W^G$ runs from the joint to the gripper origin, expressed in W because a cross product needs both operands in one frame. Any point on the axis will do as its tail: sliding along the axis adds a vector parallel to $\hat{a}_{i,W}$, which the cross product annihilates. (And note it is the gripper *origin* — the translational half of a spatial velocity is always the velocity of one named point.)

Analytic vs Geometric Jacobians

- Analytic Jacobian relates joint velocities to derivatives of pose parameters
- Geometric Jacobian relates joint velocities to spatial velocity
- The angular velocity and the derivative of orientation parameters are related by a matrix: N(q)
- The mapping between analytic and geometric Jacobians involves using this matrix to map the rotational component.

Check yourself: A single "free" body

```
plant = MultibodyPlant(time_step = 0.0)
Parser(plant).AddModelFromFile(FindResourceOrThrow(
        "drake/examples/manipulation_station/models/061_foam_brick.sdf"))
plant.Finalize()
brick = plant.GetBodyByName("base_link")

context = plant.CreateDefaultContext()
```

What size is q ? `q = plant.GetPositions(context)`

What size is v ? `v = plant.GetVelocities(context)`

For any MultibodyPlant (not just a single body):

$$\dot{q} = N(q)v$$

```
v = plant.MapQDotToVelocity(context, qdot)

qdot = plant.MapVelocityToQDot(context, v)
```

Our first Jacobian pseudo-inverse controller



$$v = [J^G(q)]^{+W} V^G$$

Moore-Penrose pseudo-inverse

https://mathformachines.com/posts/least-squares-with-the-mp-inverse/

Behavior near singularities

- Pseudo-inverse lets us handle non-square matrices
- But, near singularities, joint velocities still get very large
- One approach is damped pseudo-inverse

$$J^T(JJ^T + \lambda I)^{-1}$$

https://manipulation.csail.mit.edu/data/two_link_singularities.html?height=4 00

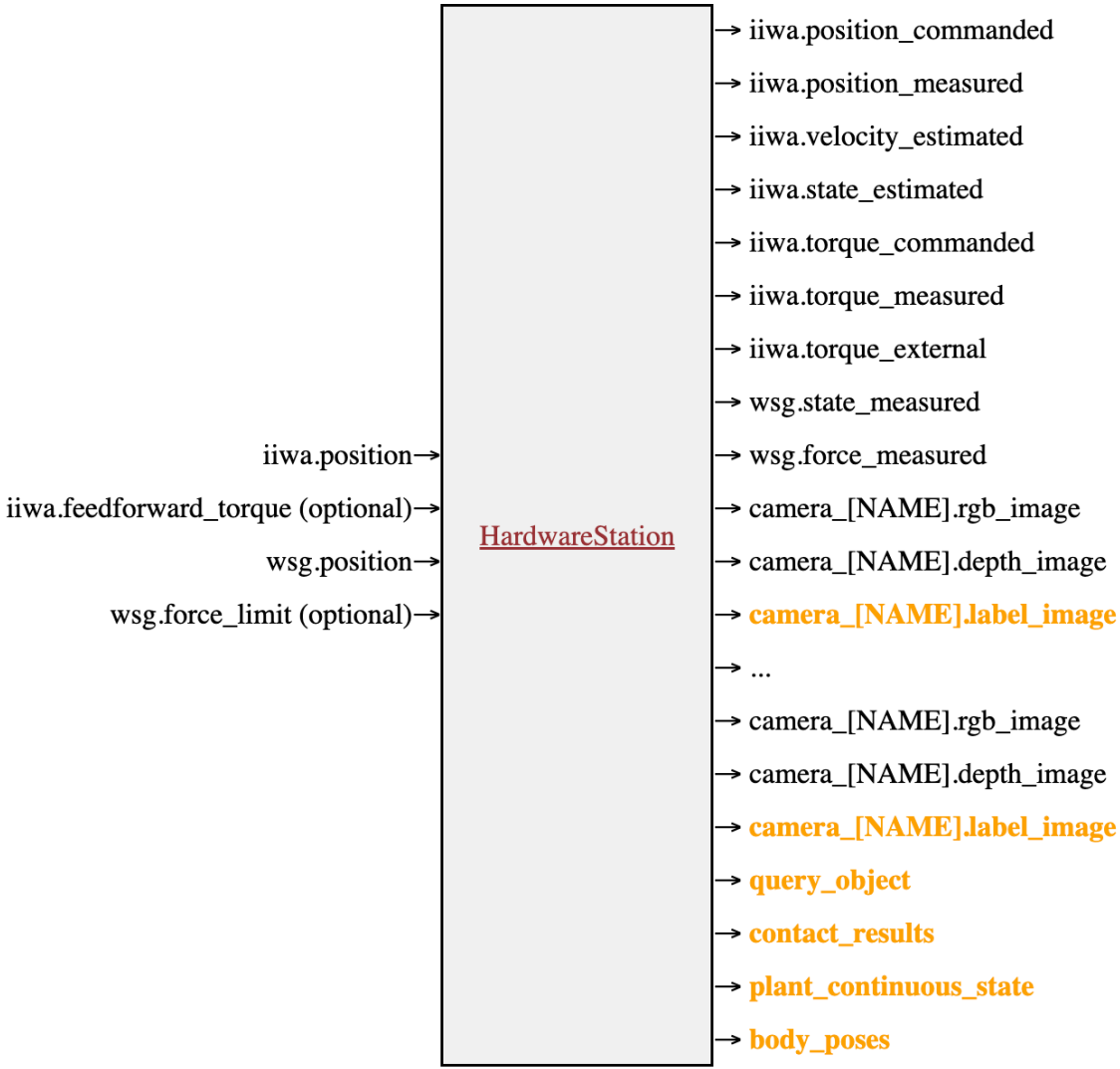
Our first Jacobian pseudo-inverse controller



$$v = [J^G(q)]^{+W} V^G$$

Our first Jacobian pseudo-inverse controller

```
1  class PseudoInverseController(LeafSystem):
2      def __init__(self, plant):
3          LeafSystem.__init__(self)
4          self._plant = plant
5          self._plant_context = plant.CreateDefaultContext()
6          self._iiwa = plant.GetModelInstanceByName("iiwa")
7          self._G = plant.GetBodyByName("body").body_frame()
8          self._W = plant.world_frame()
9
10         self.V_G_port = self.DeclareVectorInputPort("V_WG", 6)
11         self.q_port = self.DeclareVectorInputPort("iiwa.position", 7
12         self.DeclareVectorOutputPort("iiwa.velocity", 7, self.CalcOu
13         self.iiwa_start = plant.GetJointByName("iiwa_joint_1").veloc
14         self.iiwa_end = plant.GetJointByName("iiwa_joint_7").velocit
15
16     def CalcOutput(self, context, output):
17         V_G = self.V_G_port.Eval(context)
18         q = self.q_port.Eval(context)
```


iiwa.position
iiwa.feedforward_torque (optional)
wsg.position
wsg.force_limit (optional)
HardwareStation
iiwa.position_commanded
iiwa.position_measured
iiwa.velocity_estimated
iiwa.state_estimated
iiwa.torque_commanded
iiwa.torque_measured
iiwa.torque_external
wsg.state_measured
wsg.force_measured
camera_[NAME].rgb_image
camera_[NAME].depth_image
camera_[NAME].label_image
...
camera_[NAME].rgb_image
camera_[NAME].depth_image
camera_[NAME].label_image
query_object
contact_results
plant_continuous_state
body_poses

Pick and Place

```
X_O = {
    "initial": RigidTransform(
        RotationMatrix.MakeZRotation(np.pi / 2.0), [0, -0.6, 0.0]
    ),
    "goal": RigidTransform(RotationMatrix.MakeZRotation(np.pi), [0.5, 0, 0.0]),
}

builder = DiagramBuilder()

# Initialize
scenario_data = """ ... """
scenario = LoadScenario(data=scenario_data)
station = builder.AddSystem(MakeHardwareStation(scenario, meshcat=meshcat))
plant = station.GetSubsystemByName("plant")
plant.SetDefaultFreeBodyPose(plant.GetBodyByName("base_link"), X_O["initial"])

# Find the initial pose of the gripper and object (as set in the default Context)
temp_context = station.CreateDefaultContext()
temp_plant_context = plant.GetMyContextFromRoot(temp_context)
X_G = {
    "initial": plant.EvalBodyPoseInWorld(
        temp_plant_context, plant.GetBodyByName("body")
    )
}
X_O["initial"] = plant.EvalBodyPoseInWorld(
    temp_plant_context, plant.GetBodyByName("base_link")
```

Basic Pick and Place




```
prog = MathematicalProgram()
x = prog.NewContinuousVariables(2)
prog.AddConstraint(x[0] + x[1] == 1)
prog.AddConstraint(x[0] <= x[1])
prog.AddCost(x[0] ** 2 + x[1] ** 2)
result = Solve(prog)
```